모델 및 텍스쳐 파일
https://drive.google.com/file/d/1t80mXpyVaGgcFN1RhpfPVVqqEg7l9Ts0/view?usp=sharing

파일 적용 방법
- Aviator/
  - index.html
  - js/
    - game.js
    - WaterSphere.js
  - models/
  - textures/

2021204042 황인성

2025 컴퓨터 그래픽 과제3

KlingAI 2.0

게임 기획

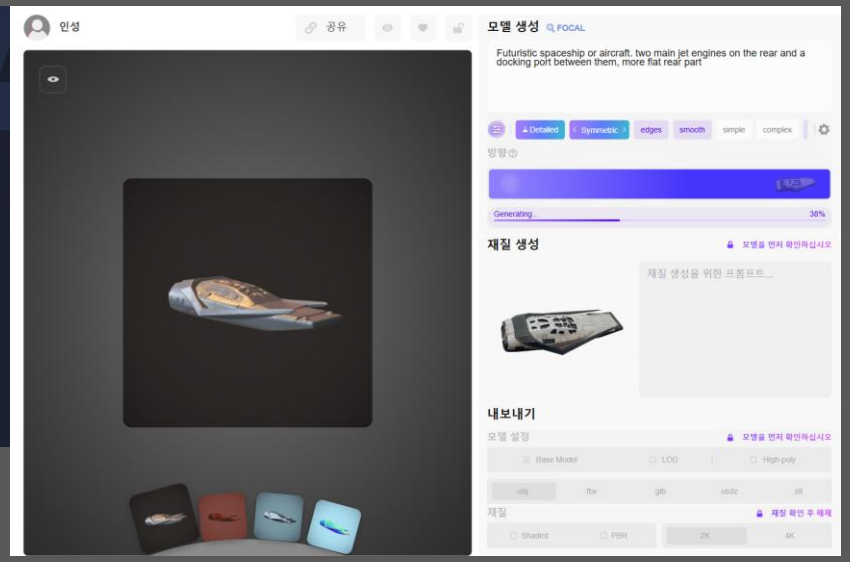
영화 ‘인터스텔라’의 한 장면을 바탕으로 기획

- 블랙홀과 매우 가까워, 매우 강한 중력으로 인한 시간 팽창과 매우 높은 파도가 치는 바다행성.

게임에 적용 :

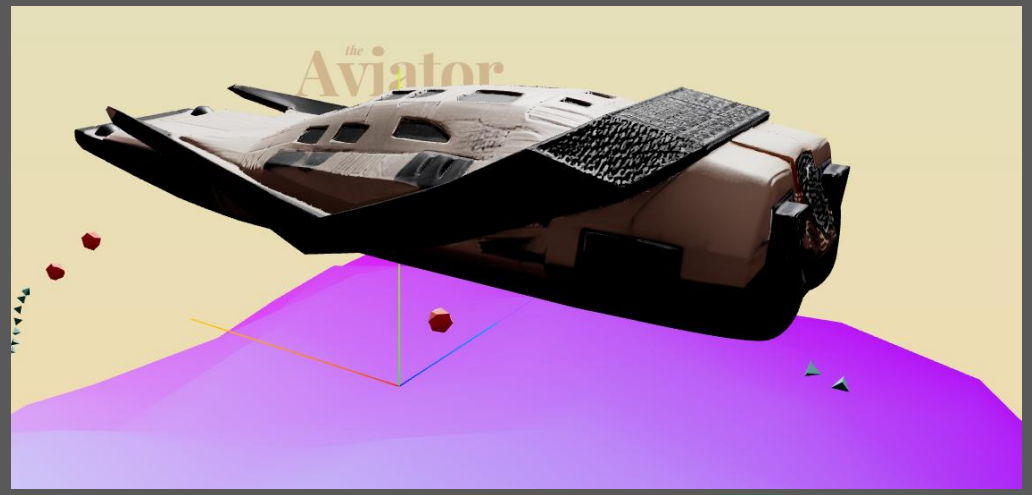
- 높은 파도가 다가오는 상황에서 한정된 연료를 사용해 파도를 건너는 게임 시나리오

우주선 외형 구현



인터스텔라의 Ranger 우주선 사진을 사용

➔ Rodin 사이트 이용 Image -> 3d Model 생성



Three js 에서 GLB 파일로 불러온 모습

우주선 인게임 요소 구현

기존 aviator 의 조종 방식을 일부 모티브로 따옴

- 마우스 상하로 우주선의 위 아래 방향 조절
  - 간단한 공기역학 적용, 비행기의 꼬리날개처럼 작동
- W 키로 가속
- A, S 키로 좌우 이동
- Space 로 자유시점 활성화
  - Fps 게임에서 주변 보기 기능

리얼한 우주선 조종 구현

- 고도에 따른 중력, 공기 저항 구현
- 가속에 따른 제트엔진 불꽃 시각화
  - 시간에 따른 불꽃 흔들림 – 커스텀 셰이더 코드로 구현
  - point light 사용 불빛 반사
- 환경맵의 태양 위치에 따른 태양광 에너지 충전
- 공기 저항 시각화 및 속도감을 위해 파티클 효과 직접 구현

바다 행성 구현

기존 cylinder 대신 Sphere 형태의 메시를 사용 -> 높은 비행기 고도에서 행성의 형태를 볼 수 있음

바다 행성 물 dynamic 패턴 texture 구현

기존 water.js 는 평면 기준으로 설계된 기능. 따라서 uv 및 반사 효과가 제대로 작동하지 않는 문제 발생

또다른 camera로 렌더링한 환경맵을 텍스쳐로 입혀서

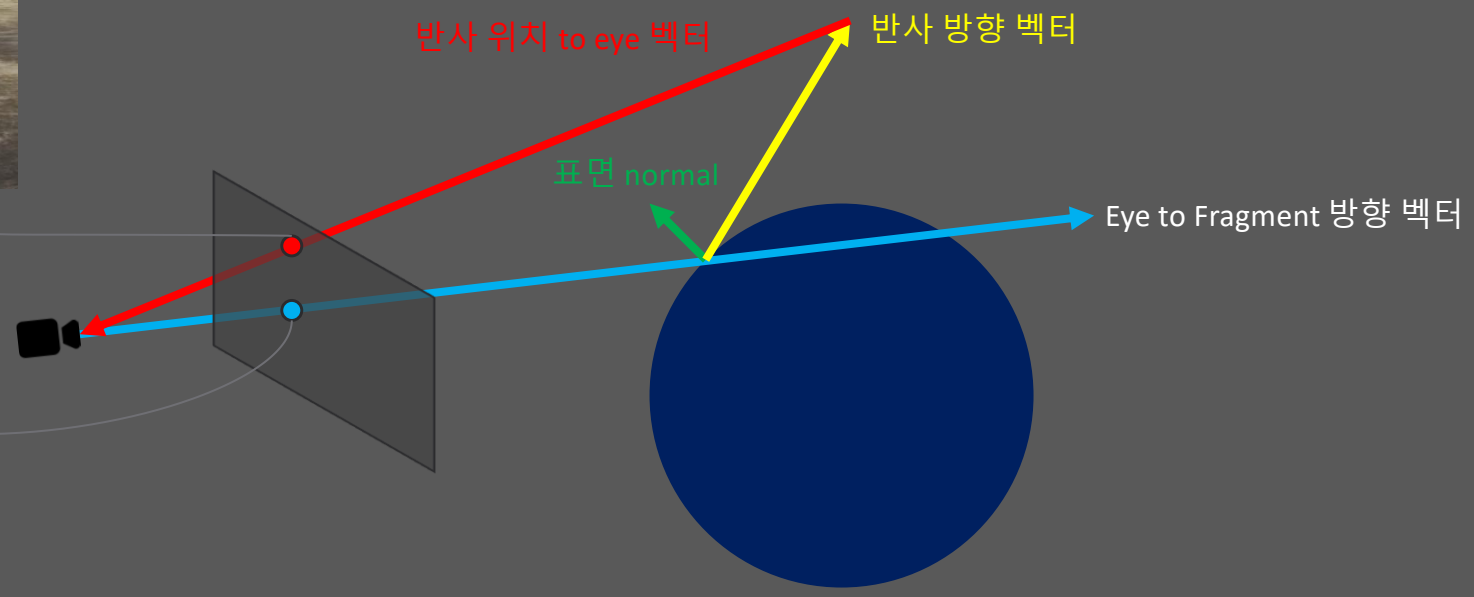
반사 직접 구현

- 문제 : 해당 카메라의 위치를 행성 표면에 위치 -> 위 사진처럼 거리에 따른 심한 왜곡 발생
- (사진에서 실제 우주선은 행성에 비해 매우 작음, 그러나 반사 텍스쳐에서는 매우 확대되어 보임)
- 반대로 카메라를 행성 중심에 위치 -> 행성 표면의 우주선 등 작은 물체들이 행성 반지름 만큼 멀리 떨어져 있는것 처럼 반사됨.

바다 행성 물 dynamic 패턴 texture 구현

구 형태에 맞춰서 Water.js 코드 수정

아이디어 : ssr 텍스쳐 샘플링 좌표를 반사를 계산한 uv 좌표로 적용하기

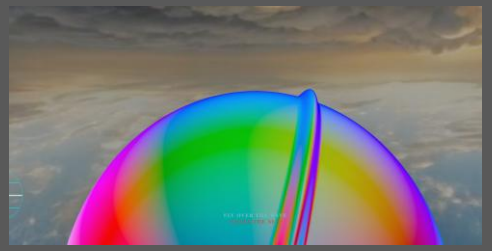


바다 행성 물 dynamic 패턴 texture 구현

구 형태에 맞춰서 Water.js 코드 수정

직접 디버깅을 통해, 기존 water.js 의 반사 텍스쳐를 구 형태 반사에 맞춰서 매핑

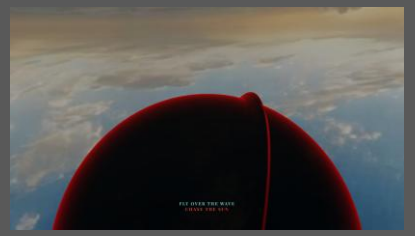
픽셀이 그리는 위치 -> 눈 벡터와 행성 Normal 내적 시각화



반사 벡터 시각화

반사 위치의 샘플링 결과

카메라 프레임 외부는 샘플링 불가-> ClampToEdgeWrapping 으로 인해 줄무니 형태 생김



Fresnel 값 시각화

반사 각도가 작을 때(물 표면과의 각도가 작을때) 보통 카메라 프레임 외부를 샘플링하게됨.

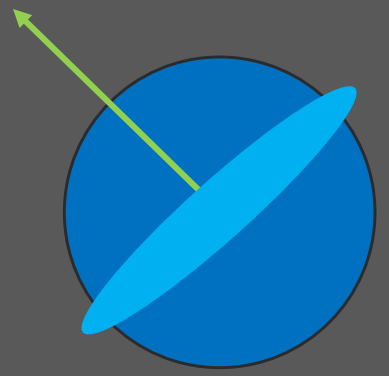
샘플링 결과 색상에 Fresnel 값을 곱해서 반사를 줄임.

거대한 파도 구현

초기 파도 방향 벡터 지정

해당 벡터와 위치 상 수직인 vertex 의 위치를 증폭해서 변경

이후 파도가 생긴 바다 행성의 bvh 적용



Loop 에서 매 틱마다 기본 파도 속도 + 비행기의 수평 속도에 비례하여 바다 행성 자체 회전

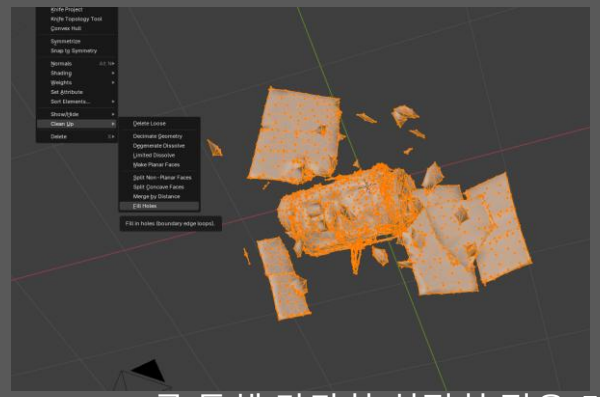
➔ 파도가 점점 다가오는 효과

버텍스의 위치를 gpu에서 계속 바꾸는 방식도 가능하지만,

이는 렌더링 결과만 바꾸고 실제 vertex 의 위치는 고정되어있으므로 bvh 가 파도에는 적용되지 않음

우주 잔해 구현

Meshy 사이트 이용 text -> 3d model 생성



Blender 를 통해 간단한 최적화 적용(메시 병합, 구멍 제거 등)

특정 y 좌표 이상에 랜덤하게 우주 잔해 배치

이후 바다가 회전해서 y 좌표가 정해진 값보다 아래로 내려간 우주 잔해의 x 좌표 반전 (다시 우주선 앞쪽으로 이동)

이를 통해 바다 전체에 우주 잔해를 채우는 대신, 우주선과 가까운 위치에만 우주 잔해 생성 및 재사용 가능

+ Bvh 사용 우주선과 충돌 감지

환경맵 구현

환경맵 구현 순서

- 2개의 큐브맵 이미지 준비 (일반 하늘, 우주)
- 큐브맵 전용 scene 를 따로 선언 후 큐브맵 메시 추가
  - 두 큐브맵의 태양 방향 일치하도록 셰이더 코드 내부에서 회전 적용
  - 고도를 uniform 으로 받아서 셰이더 코드 내부에서 두 큐브맵의 선형 보간 적용
    - 마치 행성 대기권을 벗어난 듯한 효과
- 큐브맵 메시를 우주선의 수평속도에 따라 회전
- 큐브맵 전용 scene 먼저 render 후, PMREM 이용 환경맵 생성
- 우주선, 잔해 등 물체들의 material 에 만들어진 환경맵 적용

기타

- 우주선 아래에 평면 reflector 추가
  - 직접 구현한 watersphere.js 는 정해진 거리(1000000)의 반사 위치를 가져옴. 따라서 가까이에 있는 물체 반사 불가
  - 우주선 아래에 작은 평면에 reflector 를 추가하여 우주선이 물 표면에 반사되는 것 같은 효과 구현
- Raycaster 이용 우주선이 바다 내부로 들어가는 경우까지 감지하도록 구현
  - 기존 intersectsBox는 삼각형과 box의 겹침만 감지함
  - 우주선이 바다 내부로 들어가면 감지하지 못함
  - Raycaster 를 바다 중심 방향으로 계산 – DoubleSide : false 기준 Ray 가 충돌하지 않는다면 우주선이 바다 내부에 있다고 판단 가능
  - Ray 충돌 위치를 통해 평면 reflector 의 위치까지 적용 가능
    - 구 메시가 완벽한 구가 아니라서 표면의 y 좌표가 위치에 따라 달라짐. 따라서 Reflector - 바다 표면과의 높이 차이가 발생
- css 적용한 커스텀 HUD 적용
  - 고도, 태양광 수치, 우주선의 pitch, 남은 연료, 거리를 화면에 표시

엔딩 까지 전체 플레이 영상(5분) : https://youtu.be/HnrpwmtXSQ8